UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO-COMPUTADORA

SEMESTRE 2027 - 1

PROYECTO FINAL

CEMPASÚCHIL

Ofrenda, memoria y tradición

DOCUMENTACIÓN TÉCNICA Y DE USUARIO

ALUMNOS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESOR	*Ing. José Ramón Pérez Athié*	
GRUPO	*04*	

Ciudad Universitaria, CDMX

11 de noviembre de 2026

HOJA DE EVIDENCIAS

CEMPASÚCHIL: Ofrenda, memoria y tradición

Proyecto final de Computación Gráfica e Interacción Humano-Computadora

Equipo 8 — Grupo 04

Alumno	Número de cuenta
Basilio Illescas Andrés	321137135
Cruz Macedo Samuel Santiago	424065465
Pérez Olvera Alexis Abraham	321150503
Trejo Olvera Emmanuel	424138220

Profesor: Ing. José Ramón Pérez Athié

Semestre: 2027-1

Índice

Índice de figuras 4

Índice de tablas 5

Resumen 6

1. Introducción 6

2. Concepto, memoria y significado de la ofrenda 7

2.1. Elección del Día de Muertos 7

2.2. Los siete retratos: música, fragilidad y permanencia 7

2.3. Traducción del concepto a la escena 8

3. Objetivos 8

3.1. Objetivo general 8

3.2. Objetivos específicos 8

4. Metodología práctica 9

4.1. Metodología basada en prototipos 9

4.2. Evolución del proyecto por entregas 10

4.2.1. Planeación y prototipo con primitivas 10

4.2.2. De un altar aislado a una escena completa 10

4.2.3. Reorganización e integración 11

4.2.4. Texturizado, color e iluminación 12

4.2.5. Animación e integración final 13

4.3. Organización del equipo 14

4.4. Herramientas y tecnologías 15

4.5. Modelado y composición de la escena 15

4.6. Texturizado, color e iluminación 16

4.7. Animación 17
4.7.1. Animaciones por código 17
4.7.2. Animaciones creadas en Blender 18
4.7.3. Animaciones por huesos 19
4.8. Navegación e interacción 19
4.9. Integración de las fotografías 20

5. Experimentos **21**
5.1. Compilación en distintas plataformas 21
5.2. Automatización y distribución del ejecutable para Windows 21
5.3. Control de versiones e integración del trabajo 22
5.4. Problemas de rendimiento y decisiones de optimización 23
5.5. Validación de las animaciones 23
5.6. Rendimiento 24

6. Resultados **25**

7. Evidencias visuales de la escena final **28**

8. Video demostrativo **31**

9. Conclusiones generales **31**

10. Conclusiones individuales **32**

11. Referencias **36**

Índice de figuras

1. Prototipo inicial construido con primitivas, colores planos y una composición básica de tres niveles. 10
2. Etapa de modelado: composición sin materiales (izquierda) y revisión de la construcción poligonal en modo alambre (derecha). 11
3. Etapa de texturizado e iluminación: vista de la ofrenda y detalle de materiales con mapas horneados. 12
4. Prueba de materiales translúcidos con aproximación de Fresnel para vidrio y agua. 13
5. Vista general: panteón, sendero de pétalos y casa con la ofrenda. 16
6. Vistas generales y detalles de los modelos. El recorrido conecta la entrada, el sendero y la ofrenda; los primeros planos muestran texturas horneadas, normales, transparencia y las fotografías del altar. 29
7. Iluminación interactiva y animaciones programadas en OpenGL. La tecla L modifica faroles y veladoras; las mariposas combinan Catmull–Rom con una jerarquía cuerpo–alas, y la cámara automática recorre la escena hasta el altar. 29
8. Animaciones creadas en Blender. Las cachés y pistas exportadas conservan la deformación del papel picado, el remolino de pétalos y el movimiento de la jarra con el nivel de agua. 30
9. Animaciones por huesos. Los clips muestran al visitante que deposita un pan de muerto, al personaje que recorre el panteón y al esqueleto que baila junto a la portada. 30
10. Video demostrativo de CEMPASÚCHIL. Se puede hacer clic sobre la imagen para abrir el video en YouTube. 31

Índice de tablas

1. Fases de la metodología basada en prototipos. 9
2. Roles y responsabilidades principales del equipo. 14
3. Iteraciones principales y resultado de cada incremento. 14
4. Herramientas empleadas. 15
5. Animaciones e interacciones disponibles. 17
6. Controles de la aplicación. 20

Resumen

Cempasúchil es una aplicación gráfica interactiva escrita en C++17 con OpenGL 3.3 y GLSL que recrea una noche de Día de Muertos. El visitante recorre un panteón, sigue un sendero de pétalos entre faroles, veladoras y papel picado, y llega a una ofrenda de tres niveles en el portal de una casa de adobe. El sistema reúne el pipeline gráfico completo visto en el curso: modelado jerárquico en Blender, intercambio en glTF 2.0, texturizado con mapas de color, rugosidad y normales, iluminación con un modelo de Phong y luces puntuales, transmisión con aproximación de Fresnel para vidrio y agua, un *skybox* y nueve animaciones activables con el teclado. Tres animaciones se programaron directamente en OpenGL, tres se crearon en Blender y se reproducen a partir de pistas exportadas, y tres usan personajes con esqueleto cuyos clips se evalúan con *skinning* lineal. El resultado cumple los requisitos mínimos de la especificación y se compila en macOS, Windows y Linux.

1. Introducción

El Día de Muertos es una de las celebraciones más representativas de México. La UNESCO inscribió en 2008 la festividad indígena dedicada a los muertos en la Lista Representativa del Patrimonio Cultural Inmaterial de la Humanidad (UNESCO, 2008). Sus elementos visuales (la flor de cempasúchil, el papel picado, las veladoras, la ofrenda escalonada y el camino de pétalos que guía a las almas) se prestan para un proyecto de graficación porque combinan geometría repetitiva, materiales translúcidos, fuentes de luz pequeñas y movimiento.

La especificación del curso pide un sistema interactivo que recorra todas las etapas del pipeline gráfico: conceptualización, modelado geométrico, color, texturizado, iluminación, sombreado, animación, interacción y renderizado con OpenGL 3 o superior (Segal y Akeley, 2010). Para el sombreado se emplea el modelo de reflexión de Phong (1975), que separa las componentes ambiental, difusa y especular, y la aproximación de Schlick (1994) al término de Fresnel para que vidrios y líquidos reflejen más en ángulos rasantes. Los modelos se intercambian en glTF 2.0 (Khronos Group, 2021), un formato que conserva jerarquías de nodos, materiales PBR, esqueletos y clips de animación.

En animación se usan tres familias de técnicas descritas en la literatura de tiempo real (Akenine-Möller et al., 2018; Gregory, 2018): transformaciones básicas aplicadas a una jerarquía, interpolación de *keyframes* y animación procedural. Las trayectorias se interpolan con *splines* de Catmull-Rom (Catmull y Rom, 1974), en su variante centrípeta para evitar lazos (Yuksel et al., 2011), y las rotaciones con interpolación esférica de cuaterniones (Shoemake, 1985). Los personajes con esqueleto se deforman con *skinning* lineal por vértice, la técnica

introducida por Magnenat-Thalmann et al. (1988) y que hoy es el estándar de glTF.

Este reporte describe la planeación, la evolución visual, las decisiones de diseño, los problemas técnicos, los experimentos de validación y los resultados de la versión final. El proyecto no apareció como una escena terminada: comenzó con un altar construido con cubos, cilindros, esferas y colores planos; creció hasta convertirse en un entorno completo; recibió texturas, iluminación y materiales; y finalmente incorporó navegación y nueve animaciones interactivas. Documentar ese proceso permite explicar no sólo qué se entregó, sino por qué se tomó cada decisión.

2. Concepto, memoria y significado de la ofrenda

2.1. Elección del Día de Muertos

El equipo eligió el Día de Muertos porque ofrece una unión natural entre memoria, recorrido y representación visual. No se quería construir un altar aislado que el usuario observara desde una sola posición, sino una experiencia: entrar por un panteón, avanzar entre tumbas, seguir la luz y los pétalos, y descubrir gradualmente la ofrenda. De esta manera, la navegación también forma parte del relato. El sendero de cempasúchil orienta al visitante; las veladoras y los faroles hacen visible el camino; el papel picado introduce color y movimiento; las mariposas monarca evocan el retorno; y el altar concentra los alimentos, recipientes, flores y retratos.

La escena evita tratar la muerte solamente como oscuridad. La noche, el panteón y las tumbas establecen el contexto, pero el color naranja, la luz cálida, el movimiento de los pétalos y la presencia de las ofrendas transforman el espacio en un sitio de encuentro. Esta dualidad guió la paleta: azules fríos y sombras en el recorrido, frente a naranjas, magentas, amarillos y luz cálida al acercarse al altar.

2.2. Los siete retratos: música, fragilidad y permanencia

La versión final dedica sus siete portarretratos a Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix y Kristen Pfaff. Los siete murieron a los 27 años. Sus trayectorias pertenecen a géneros, países y generaciones diferentes, pero comparten una intensidad creativa que sobrevivió a una vida breve.

La intención no es romantizar los excesos ni convertir el sufrimiento en una condición del talento. Por el contrario, la ofrenda recuerda que detrás de las figuras públicas existieron personas sometidas a presiones, aislamiento, adicciones, violencia, agotamiento y expecta-

tivas que con frecuencia fueron más grandes que los apoyos disponibles. Valentín Elizalde representa además una tradición musical mexicana y una muerte atravesada por la violencia; Kristen Pfaff permite reconocer también a quienes, aun sin ocupar siempre el centro del escenario, dejaron una huella decisiva en su generación.

El altar propone un gesto de memoria: separar por un momento a estas personas del mito de los "27" y devolverles un lugar individual. Cada fotografía es distinta y ocupa una posición propia. Alrededor de ellas aparecen agua, pan, flores, velas, comida y objetos cotidianos. En conjunto, la escena expresa que una obra puede permanecer aun cuando una vida haya sido corta y difícil, y que recordar también implica mirar con empatía los costos humanos que hubo detrás de esa obra.

2.3. Traducción del concepto a la escena

La distribución espacial refuerza el significado. Los retratos se reparten en los tres niveles para evitar una jerarquía entre los homenajeados. El arco de cempasúchil enmarca el conjunto; las veladoras forman líneas y grupos que conducen la mirada; el agua y los recipientes aprovechan materiales translúcidos; el pan y los alimentos aportan volumen y variedad; y el papel picado rompe la rigidez de los escalones. Una mariposa descansa frente a la ofrenda y puede emprender el vuelo, mientras otras aparecen a lo largo del camino. La escena completa funciona así como una transición desde el silencio del panteón hasta una memoria iluminada y en movimiento.

3. Objetivos

3.1. Objetivo general

Desarrollar un sistema gráfico interactivo en OpenGL 3.3 que represente una ofrenda de Día de Muertos y aplique el pipeline gráfico completo, desde el modelado hasta la animación interactiva.

3.2. Objetivos específicos

- Integrar al menos ocho modelos propios y ocho modelos de librería, texturizados y organizados en un grafo de escena jerárquico.

- Iluminar la escena con al menos dos técnicas (Phong para materiales opacos y Fresnel para materiales translúcidos) y un mapa de entorno.
- Ofrecer al menos dos interacciones con luces en tiempo real.
- Integrar tres animaciones por código, tres creadas en software externo y tres animaciones por huesos, todas activables con el teclado.
- Mantener la compilación con CMake en macOS, Windows y Linux.

4. Metodología práctica

4.1. Metodología basada en prototipos

El trabajo siguió las fases de la metodología basada en prototipos de la especificación. En la recolección de requisitos se eligió el tema, se definieron las interacciones y se acotó el alcance de cada entrega. El diseño conceptual produjo la distribución de la escena (panteón al norte, sendero al centro y casa con ofrenda al sur) y el catálogo de modelos. A partir de ahí se construyó un primer prototipo con geometría gris, que en cada entrega se probó, se corrigió y se amplió: primero con modelado completo, después con texturas e iluminación, y al final con animación. La Tabla 1 resume cómo se recorrió cada fase.

Fase	**Actividad en el proyecto**
Requisitos y planificación	Tema “México y su cultura” con la ofrenda de Día de Muertos; lista de modelos, luces e interacciones mínimas.
Diseño conceptual	Plano de la escena, eje del sendero como *spline* y flujo de navegación en primera persona.
Prototipo inicial	Ventana GLFW, carga glTF, cámara y grafo de escena con geometría sin textura.
Validación y retroalimentación	Revisión del equipo en cada entrega parcial; ajustes de escala, intersecciones y rendimiento.
Iteración y mejoras	Texturas PBR, luces puntuales, filtrado de luces, *skybox* y animaciones.
Implementación completa	Integración de las nueve animaciones con entrada por teclado y documentación.

Tabla 1: Fases de la metodología basada en prototipos.

4.2. Evolución del proyecto por entregas

4.2.1. Planeación y prototipo con primitivas

La primera versión tuvo un alcance deliberadamente pequeño. Se construyó un altar de tres niveles con cubos y se representaron las ofrendas mediante primitivas: cilindros para velas, esferas para frutas, un toro y un cono para una flor simplificada, y placas planas para anticipar los retratos. Los materiales eran colores planos y una luz cálida permitía distinguir los volúmenes. Este prototipo no buscaba realismo; servía para responder preguntas de composición: cuántos niveles debía tener el altar, qué proporción ocuparía cada uno, dónde se concentraría la mirada y cuánto espacio necesitarían las ofrendas futuras.

La principal enseñanza de esta etapa fue que una representación sencilla puede revelar problemas antes de invertir tiempo en detalle. Las primitivas permitieron mover grupos enteros, probar el encuadre y confirmar que el altar escalonado ofrecía suficientes superficies para distribuir objetos sin perder legibilidad.

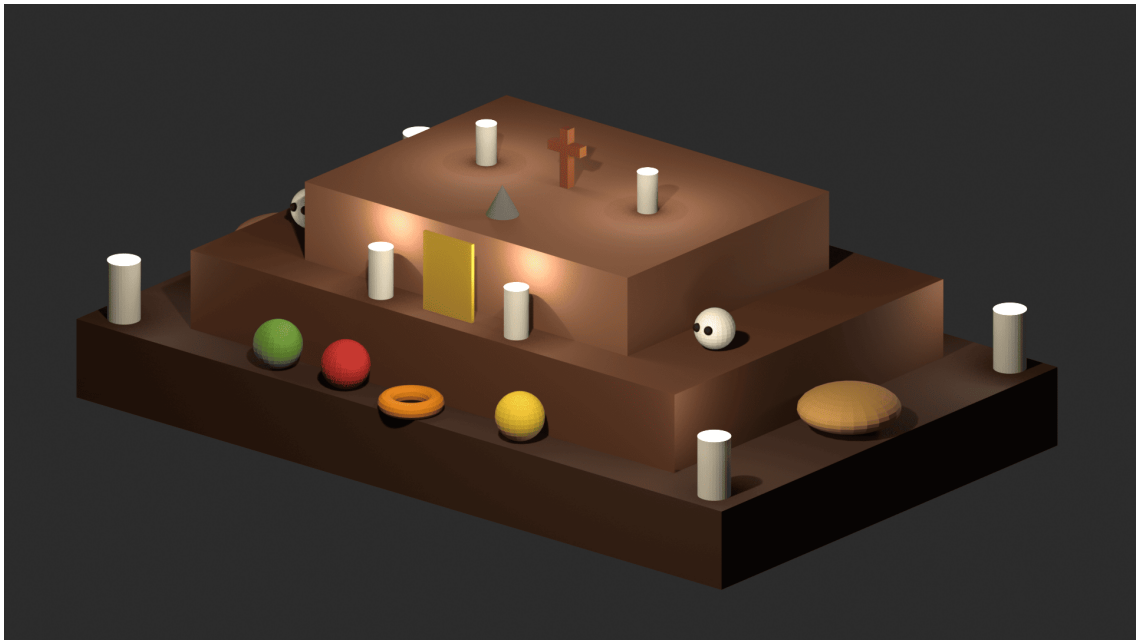


Figura 1: Prototipo inicial construido con primitivas, colores planos y una composición básica de tres niveles.

4.2.2. De un altar aislado a una escena completa

Durante la entrega de modelado se concluyó que el altar, aunque reconocible, no bastaba para comunicar una experiencia. Se amplió el alcance hacia un entorno recorrible: una casa de adobe con portal, un sendero central y un panteón al extremo opuesto. El trayecto permitió

distribuir elementos que no cabían de manera natural en la ofrenda, como faroles, papel picado suspendido, tumbas, criptas, bardas, rejas, vegetación y mariposas.

Los primeros modelos propios se revisaron y, cuando fue necesario, se remodelaron. Se corrigieron escalas, orígenes, normales, densidad de malla y proporciones. Algunos objetos que funcionaban de cerca resultaban demasiado pesados al repetirse; otros eran demasiado simples cuando se colocaban en primer plano. El equipo adoptó una definición de terminado para cada modelo: transformaciones aplicadas, origen cerca de la base, nombre identificable, normales correctas, complejidad razonable, exportación glTF válida y verificación tanto sólida como en alambre.

La meta mínima de ocho modelos propios se superó al repartir el trabajo por familias. Se modelaron el altar, flores, pétalos, veladoras, pan, calaveritas, papel picado, sahumerio, portarretratos, jarrones, recipientes, mariposas, faroles, arquitectura y, posteriormente, alimentos, dulces y botellas. A la vez, los elementos ambientales del *Kenney Graveyard Kit* complementaron el panteón sin sustituir la autoría de los objetos principales.

Figura 2: Etapa de modelado: composición sin materiales (izquierda) y revisión de la construcción poligonal en modo alambre (derecha).

4.2.3. Reorganización e integración

Al aumentar el catálogo, la dificultad dejó de ser solamente modelar y pasó a ser organizar. Los objetos debían compartir una escala coherente, apoyarse correctamente sobre superficies y no invadir el espacio de otros. La ofrenda se reorganizó por zonas y niveles; se reservaron celdas para grupos de veladoras, alimentos, retratos y recipientes, y se aplicaron comprobaciones geométricas para evitar intersecciones. En el sendero se utilizó una curva Catmull–Rom como eje común para repartir pétalos y orientar objetos laterales.

Esta etapa motivó el grafo de escena y los lotes instanciados. Una transformación aplicada a un nodo padre mueve todas sus piezas, mientras que miles de elementos repetidos comparten

una sola geometría. La arquitectura hizo posible seguir ampliando la escena sin convertir cada copia en un modelo independiente.

4.2.4. Texturizado, color e iluminación

La entrega siguiente transformó la lectura de la escena. Los tonos grises fueron sustituidos por materiales que diferencian adobe, cera, papel, metal, cerámica, vidrio, agua, flores y alimentos. Para evitar depender completamente de fotografías externas, muchas texturas se diseñaron de forma procedural en Blender. Redes de ruido, mezclas de color y relieves se hornearon a mapas convencionales de color, rugosidad y normales que glTF y OpenGL pudieran cargar con eficiencia.

El horneado resolvió una diferencia importante entre Blender y el motor propio: una red nodal compleja no se reproduce automáticamente en GLSL. Convertirla a imágenes permitió conservar su apariencia general y controlar el costo en tiempo real. Fue necesario corregir UV, márgenes, espacios de color y orientación de mapas normales, además de reducir mallas cuya densidad ya no aportaba detalle visible.

Figura 3: Etapa de texturizado e iluminación: vista de la ofrenda y detalle de materiales con mapas horneados.

La iluminación también se rediseñó. Una luz direccional fría establece la noche, mientras que veladoras y faroles generan luz puntual cálida. El altar recibe rellenos agrupados para conservar su presencia sin evaluar cada fuente contra todos los objetos. Los materiales de vidrio y agua emplean Fresnel para reforzar los contornos y transmitir el fondo. La decisión final fue mantener toda la luz en OpenGL: no se horneó iluminación estática en las texturas, de modo que el encendido, apagado y parpadeo permanecen interactivos.

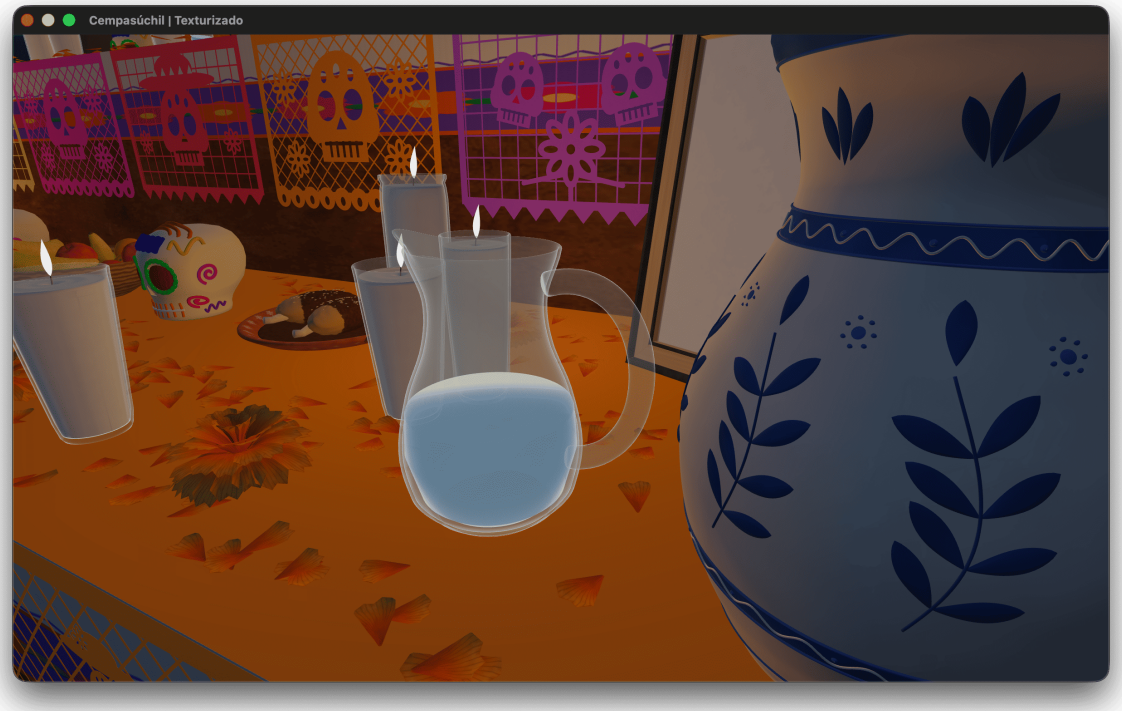


Figura 4: Prueba de materiales translúcidos con aproximación de Fresnel para vidrio y agua.

4.2.5. Animación e integración final

Con la escena modelada, texturizada e iluminada, la última fase incorporó movimiento sin reemplazar el motor existente. Se produjeron tres animaciones por código, tres en Blender y tres con huesos. Primero se trabajaron como clips o pruebas aisladas; después se integraron a la escena, se asignaron teclas, se adaptaron a las jerarquías existentes y se optimizaron para no invalidar el filtrado de luces ni el descarte por cámara.

El paso más delicado fue reproducir fielmente animaciones externas. Una exportación convencional no siempre conserva modificadores, simulaciones o *shape keys* como se ven en Blender. Por ello, el papel picado se convirtió en una caché de posiciones por vértice; el remolino y la jarra se exportaron como pistas binarias; y los personajes se exportaron en glTF con armature, *skin* y clips. La versión final reproduce esos datos dentro de OpenGL y los combina con la interacción y la iluminación del motor.

4.3. Organización del equipo

El equipo adaptó Scrum a las cuatro entregas de la asignatura. Cada entrega funcionó como un sprint con un incremento ejecutable: prototipo, modelado, texturizado e iluminación, y animación con cierre del producto. El Product Backlog reunió tareas de infraestructura, modelado, composición, materiales, animación, optimización y documentación. Al iniciar cada iteración se priorizaron las tareas por dependencia y valor para la escena; durante el desarrollo se comunicaron avances y bloqueos, y al cierre se realizó una revisión completa en OpenGL y una retrospectiva breve.

Integrante	**Rol**	**Responsabilidad principal**
Basilio Illescas Andrés	Product Owner	Priorización del alcance, validación cultural y revisión de incrementos.
Trejo Olvera Emmanuel	Scrum Master	Coordinación, integración Blender–OpenGL, rendimiento y compatibilidad.
Pérez Olvera Alexis Abraham	Developer	Modelos de la ofrenda, materiales, texturas y exportaciones glTF.
Cruz Macedo Samuel Santiago	Developer	Arquitectura, panteón, recorrido y pruebas de integración.

Tabla 2: Roles y responsabilidades principales del equipo.

El seguimiento utilizó los estados *pendiente*, *en desarrollo*, *en revisión*, *integrado* y *terminado*. Una tarea sólo se cerraba cuando estaba incorporada en la escena principal y superaba sus criterios de aceptación. Para un modelo se revisaban escala, origen, normales, UV, complejidad y exportación; para código, compilación limpia, rutas portables y ausencia de regresiones; y para una animación, activación por entrada, reproducción estable e integración visual. Las ramas y commits separaron los cambios, mientras que `.gitignore` excluyó compilaciones y temporales.

Sprint	**Objetivo**	**Incremento verificable**
1	Prototipo	Concepto, altar con primitivas, cámara y distribución inicial.
2	Modelado	Entorno recorrible, modelos propios, jerarquías e instancias.
3	Apariencia	Texturas, skybox, Phong, Fresnel, luces y optimización inicial.
4	Animación y cierre	Nueve animaciones, navegación, pruebas, documentación y empaquetado.

Tabla 3: Iteraciones principales y resultado de cada incremento.

Los riesgos principales fueron la incompatibilidad entre plataformas, modelos demasiado pesados, exceso de luces, animaciones externas difíciles de reproducir y conflictos de Git sobre archivos binarios. Se mitigaron con integraciones parciales, pruebas en macOS, Windows

y Linux, horneado de datos de animación, instanciado, filtrado de luces y conservación de versiones estables. Esta organización mantuvo unido el trabajo de Blender, OpenGL y documentación sin convertir las entregas parciales en proyectos separados.

4.4. Herramientas y tecnologías

Herramienta	Uso
C++17	Aplicación, grafo de escena, interacción y animación.
OpenGL 3.3 Core y GLSL 330	Renderizado, materiales, iluminación y recorte por plano.
Blender 5.2 (Blender Foundation, 2026)	Modelado, horneado de texturas, animación de papel, pétalos y jarra, y esqueletos de personajes.
glTF 2.0 y cgltf	Intercambio de modelos, materiales, esqueletos y clips.
GLFW, GLAD y GLM	Ventana y entrada, carga de funciones OpenGL y matemáticas.
stb_image	Lectura de PNG y JPEG en Windows y Linux.
CMake	Configuración multiplataforma.

Tabla 4: Herramientas empleadas.

4.5. Modelado y composición de la escena

Los modelos propios se construyeron en Blender a partir de primitivas con extrusión, superficies de revolución y modificadores, y se exportaron a glTF conservando su jerarquía. Así, el altar se descompone en sus tres niveles y el farol en poste, brazo y lámpara. La aplicación replica esa jerarquía como nodos editables: cada nodo guarda traslación, rotación y escala locales, y su matriz de mundo se obtiene como $M_{\text{mundo}} = M_{\text{padre}}\, T\, R\, S$.

La escena final tiene 2 134 nodos, 81 modelos cargados y 10 549 instancias. Los elementos que se repiten miles de veces (pétalos, veladoras, flores y bardas) se agrupan en lotes y se dibujan con `glDrawElementsInstanced`, con una matriz y un tinte por instancia. El sendero es una *spline* Catmull-Rom centrípeta, evaluada con el algoritmo de Barry y Goldman (1988) y reparametrizada por longitud de arco; con ella se colocan pétalos, veladoras, faroles y papel picado a distancias regulares. La Figura 5 muestra la distribución.

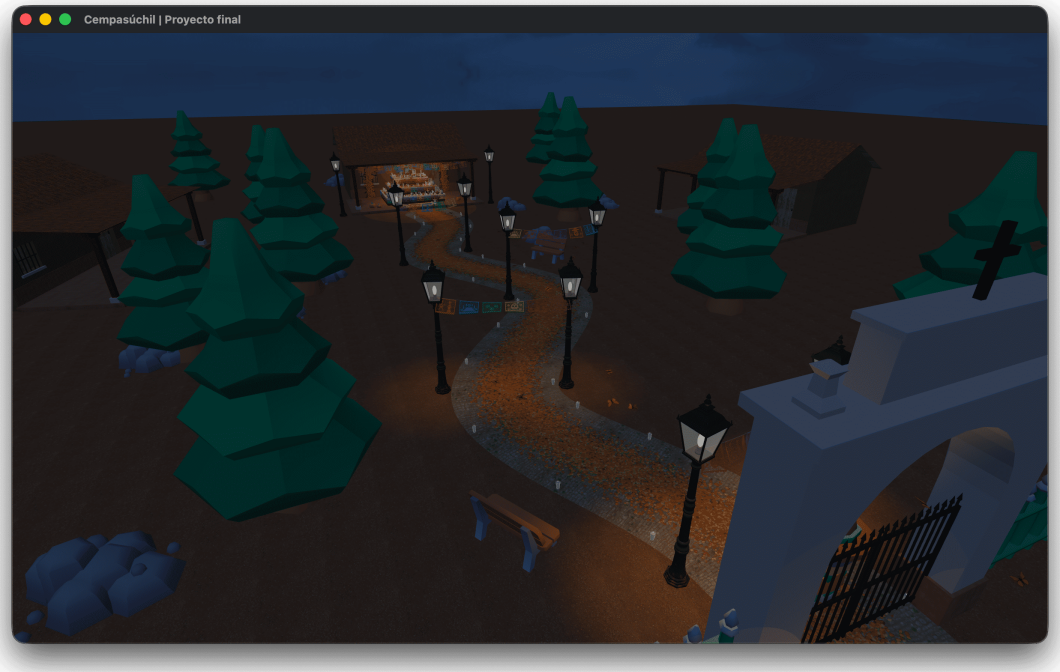


Figura 5: Vista general: panteón, sendero de pétalos y casa con la ofrenda.

4.6. Texturizado, color e iluminación

Las texturas de los modelos propios se diseñaron con nodos procedurales en Blender y se hornearon a mapas de color, rugosidad y normales. En el *fragment shader* las normales se perturban sin tangentes precalculadas: las derivadas `dFdx` y `dFdy` de la posición y de las coordenadas UV dan la base tangente de cada píxel. Los pétalos y el papel picado usan recorte alfa para mostrar su silueta sin geometría adicional.

La iluminación combina dos técnicas:

- **Phong** para materiales opacos: una luz direccional lunar azulada y hasta 25 luces puntuales seleccionadas entre veladoras, faroles y rellenos del altar, con atenuación cuadrática. El brillo especular se deriva de la rugosidad del material.
- **Fresnel** para vidrio y agua: la aproximación de Schlick mezcla el color transmitido con un reflejo que crece en ángulos rasantes. Así se ven los vasos, las jarras y las veladoras.

El entorno es un *skybox* con mapa cúbico. Su *vertex shader* elimina la traslación de la vista para que el horizonte no se mueva, y su *fragment shader* oscurece el atardecer original hacia un tono nocturno. Para no recorrer luces inútiles, el renderizador filtra por caja envolvente

las luces que pueden alcanzar a cada objeto y descarta las primitivas fuera de cámara con los planos del *frustum* extraídos de la matriz de vista y proyección (Gribb y Hartmann, 2001).

4.7. Animación

La especificación pide animaciones de tres tipos (básica, por *keyframes* y procedural) y tres medios (código, software externo y huesos). La Tabla 5 relaciona cada animación con su tecla, su medio y su técnica. Todas se coordinan desde la clase `Animaciones` (`src/scene/Animaciones.cpp`). Esta clase recibe la tecla, avanza cada animación con el tiempo del cuadro y escribe el resultado en los nodos del grafo de escena o en los *uniforms* del renderizador.

Tecla	Animación	Medio	Técnica
1	Mariposa monarca	Código	Máquina de estados, *keyframes* sobre Catmull-Rom y aleteo procedural en la jerarquía cuerpo-alas.
2	Recorrido de cámara	Código	Siete *keyframes* de posición y mira con interpolación suave.
3	Veladoras	Código	Estados de encendido y apagado; encendido por grupos y parpadeo procedural en GLSL.
4	Papel picado	Blender	*Shape keys* horneadas en una caché de vértices de 96 cuadros.
5	Remolino de pétalos	Blender	*Keyframes* de 720 pétalos exportados como pistas.
6	Jarra sirviendo agua	Blender	*Keyframes* de la jarra, nivel del agua y chorro exportados como pistas.
7	Visitante 1	Huesos	*Armature* de siete huesos; caminata, depósito de la ofrenda y reverencia.
8	Visitante 2	Huesos	Segundo personaje; observa una tumba, saluda y baila.
9	Esqueleto bailando	Huesos	Baile de 12 s con cinco pasos mezclados.
L	Faroles	Código	Interacción de luz: apagado y encendido gradual.

Tabla 5: Animaciones e interacciones disponibles.

4.7.1. Animaciones por código

Mariposa monarca (1). Una mariposa reposa sobre el petate frente al altar. Al presionar la tecla, recorre la máquina de estados reposo, despegue, vuelo y aterrizaje. La ruta se define con diez puntos de control que pasan por el reposo, suben frente al arco y regresan; se interpola con la misma clase `Camino` del sendero y se recorre a velocidad constante. Los estados se

distinguen por la distancia recorrida: en los primeros y últimos 0.6 m el aleteo crece o se apaga, y en el tramo central alcanza su amplitud completa. Las alas son nodos hijos del cuerpo, de modo que basta rotarlas alrededor de su bisagra para que acompañen la traslación y la orientación del cuerpo. Es un ejemplo de jerarquía de transformaciones.

Recorrido de cámara (2). Siete *keyframes* de posición y siete de punto de mira describen la entrada desde el panteón, el cruce de la portada, el sendero y la llegada a la ofrenda. Ambas secuencias se interpolan con Catmull-Rom y comparten un parámetro de tiempo suavizado con $3t^2 - 2t^3$, de modo que la cámara arranca y se detiene sin saltos. Mientras dura el recorrido se ignora la entrada de movimiento; la misma tecla lo interrumpe.

Veladoras (3). Una máquina de estados en C++ pasa por encendidas, apagando, apagadas y encendiendo. Cada estado fija dos valores que llegan al *shader*: un frente de encendido sobre el eje Z y un nivel global. En GLSL, la función `factorVela` agrupa las velas en tramos de 4.4 m. Cada tramo enciende cuando el frente lo alcanza, así que las velas se prenden por grupos desde el panteón hasta el altar. El mismo factor agrega un parpadeo procedural: dos senoidales con una fase pseudoaleatoria por vela. El factor modula tanto la emisión de las flamas como la luz puntual que cada vela proyecta sobre su entorno.

Faroles (L). La segunda interacción de luz reduce gradualmente a cero la emisión y la luz de los faroles, y la restablece al presionar de nuevo.

4.7.2. Animaciones creadas en Blender

Las tres animaciones de software externo se modelaron y animaron en Blender. Para el motor se hornearon en datos que OpenGL reproduce sin depender de Blender.

Papel picado (4). Seis pliegos se deforman con *shape keys* (borde fijo, curvatura y onda) animadas por *keyframes* con fases y amplitudes distintas. El resultado se horneó como caché de vértices de 96 cuadros; en cada cuadro el motor copia las posiciones al búfer de vértices. La tecla calma o reanuda el viento: la velocidad de reproducción baja o sube de forma gradual.

Remolino de pétalos (5). En Blender, un *script* calculó los *keyframes* de posición y rotación de 720 pétalos: se levantan del sendero, giran en una espiral de dos brazos frente al altar y se asientan con vaivén. El *script* `exportar_remolino.py` lee la escena cuadro por cuadro y guarda, por pétalo, su posición, su cuaternión y su escala en el archivo binario `remolino_petalos.pist`. Durante la conversión cambia los ejes de Blender (Z arriba) a los de OpenGL (Y arriba). En el motor los pétalos forman un lote instanciado. Cada cuadro se interpolan los dos cuadros vecinos de la pista y se reescriben las 720 matrices con `glBufferSubData`.

Jarra sirviendo agua (6). La jarra gira alrededor de un pivote en su pico. El agua interior se recorta con una caja cuya altura está animada para que la superficie se mantenga horizontal y el volumen se conserve; el chorro es una curva con bisel animado; el vaso se llena con una *shape key*. El *script* `exportar_jarra.py` exporta la posición y rotación de la jarra, la altura de la superficie, el llenado del vaso y los factores del chorro. En OpenGL el recorte del agua se resuelve con un plano: el *fragment shader* descarta los fragmentos con $\mathbf{n} \cdot \mathbf{p} + d > 0$. El chorro es un tubo cuyos anillos se recolocan sobre la curva exportada en cada cuadro.

4.7.3. Animaciones por huesos

Los personajes parten de las piezas del *Kenney Graveyard Kit* (Kenney, 2021), con licencia CC0. El *script* `exportar_personajes.py` crea en Blender un *armature* de siete huesos (cadera, torso, cabeza, brazos y piernas), une las piezas en una malla y liga cada pieza a su hueso. Después genera los clips (reposo, caminar, caminar cargando, depositar, incorporarse, reverencia, observar, saludar, baile corto y baile completo) y exporta cada personaje como glTF con *skin* y una animación por clip.

En el motor, `Modelo::posar` interpola las pistas de traslación y rotación de cada hueso (lineal y esférica), calcula sus matrices globales y aplica *skinning* lineal:

$$\mathbf{v}' = \sum_i w_i \, G_i \, B_i^{-1} \, \mathbf{v}, \tag{1}$$

donde G_i es la matriz global del hueso, B_i^{-1} su matriz inversa de *bind* y w_i el peso del vértice. Los vértices deformados se suben al búfer de la malla y la caja envolvente se recalcula para el descarte por *frustum*. Entre clips hay una mezcla de 0.25 s para evitar saltos de pose.

La trayectoria de cada personaje se describe en código como una secuencia de tramos (caminar por una ruta, girar, ejecutar un clip). Al caminar, la fase del ciclo avanza con la distancia recorrida (0.95 m por ciclo), lo que evita que los pies patinen. El visitante 1 lleva un pan de muerto colgado de un nodo ancla hijo del antebrazo; al terminar el clip de depósito, el pan queda sobre el primer piso del altar.

4.8. Navegación e interacción

La navegación se diseñó como un recorrido en primera persona. Al iniciar, la cámara aparece en la entrada del panteón, a una altura aproximada de 1.3 m sobre el suelo y orientada hacia el altar. Esta posición ofrece al usuario un objetivo visual desde el primer cuadro: la iluminación y el camino de pétalos indican hacia dónde avanzar sin recurrir a una interfaz superpuesta.

En el modo de caminata, WASD desplaza al visitante sobre el plano del suelo. La altura se mantiene fija y se suma un balanceo vertical y lateral de baja amplitud. La frecuencia y la suavidad se ajustaron para sugerir pasos sin provocar una oscilación agresiva. Shift multiplica temporalmente la velocidad y la rueda del ratón modifica la velocidad base. La orientación se realiza con el movimiento del ratón; TAB captura o libera el cursor, lo cual permite alternar entre control directo y uso normal del escritorio.

La tecla F activa el vuelo libre. Solamente en ese modo se habilitan Q y E para descender y ascender. Esta separación evita que el visitante abandone accidentalmente el nivel del suelo durante el recorrido normal, pero conserva una herramienta útil para inspeccionar la escena, grabar evidencias o revisar objetos desde arriba. R devuelve la cámara a la entrada y F2 alterna entre el acabado final y la malla de alambre.

Además del movimiento libre, la tecla 2 inicia un recorrido de cámara preparado como demostración. Durante ese clip la entrada de desplazamiento se suspende para impedir que el usuario desplace la trayectoria; al volver a presionar la tecla, el recorrido se cancela y el control regresa inmediatamente. La Tabla 6 resume todos los controles.

Entrada	Acción
W, A, S, D	Caminar por la escena.
Shift	Aumentar la velocidad.
Ratón y TAB	Orientar la cámara; TAB activa o libera el ratón.
F, Q, E	Alternar vuelo libre; descender y ascender en vuelo.
Rueda del ratón	Ajustar la velocidad de desplazamiento.
1 a 9	Iniciar las animaciones de la Tabla 5.
L	Apagar o encender los faroles.
F2	Alternar sólido y malla de alambre.
R	Regresar a la entrada del panteón.
Esc	Cerrar la aplicación.

Tabla 6: Controles de la aplicación.

4.9. Integración de las fotografías

Los siete portarretratos muestran fotografías diferentes. Para evitar duplicar siete modelos glTF completos, el motor carga tres portarretratos base y crea variantes lógicas del material de la superficie fotográfica. Cada variante sustituye solamente la textura de esa superficie y conserva el marco, la geometría, los mapas y el resto de los materiales. Las imágenes se preprocesaron a JPEG, con orientación y encuadre compatibles con las coordenadas UV del marco, y se almacenan en assets/textures/fotografias/procesadas/.

Esta solución permitió asignar a cada posición una persona distinta sin reexportar el altar ni

modificar la composición aprobada. Tres retratos ocupan el nivel superior y cuatro se colocan detrás de los jarrones de los niveles primero y segundo. La selección del material por nombre mantiene el cambio acotado a la fotografía: no altera la madera, el vidrio o la iluminación del marco.

5. Experimentos

5.1. Compilación en distintas plataformas

El proyecto se compiló con CMake en macOS (Xcode), Windows (Visual Studio 2022 y 2026) y Linux (GCC). Las pruebas de las entregas anteriores encontraron problemas que siguen resueltos en esta versión:

- Fuera de macOS no aparecían las texturas porque la decodificación de imágenes solo existía con ImageIO; se integró `stb_image` para Windows y Linux.
- En pantallas con escalado, el tamaño lógico de la ventana no coincide con el del *framebuffer*, así que se usa `glfwGetFramebufferSize` antes de `glViewport`.
- `Rutas.cpp` localiza `assets/` y `shaders/` desde la ruta real del ejecutable, con rutas anchas en Windows, para no depender del directorio de trabajo.
- Las diferencias de redondeo entre compiladores cambian en unas pocas piezas el conteo de la ofrenda; una diferencia pequeña se registra como aviso.
- El controlador VirtIO de una máquina virtual con Windows 11 ARM no expuso OpenGL 3.3, por lo que la aplicación requiere un controlador con OpenGL 3.3 real.

5.2. Automatización y distribución del ejecutable para Windows

Para producir una entrega reproducible sin depender de que un integrante compilara manualmente en Windows, se incorporó el flujo `.github/workflows/windows-installer.yml`. El proceso puede iniciarse manualmente desde GitHub Actions y también se ejecuta al publicar una etiqueta cuyo nombre comienza con `v`. GitHub asigna una máquina virtual con Windows Server 2022; en ella se configura el proyecto con CMake y Visual Studio 2022 para arquitectura de 64 bits, se compila la configuración *Release* y se verifica que existan el ejecutable, los modelos, las texturas, las animaciones, los *shaders* y los archivos de instrucciones.

El empaquetado se realiza con CPack y NSIS. Las reglas de instalación definidas en `CMakeLists.txt` incorporan `CEMPASUCHIL.exe`, las carpetas `assets/` y `shaders/`, el archivo `README.txt` y la licencia en un solo instalador. Las bibliotecas GLFW y GLAD se integran durante la compilación y el tiempo de ejecución de Visual C++ se enlaza de forma estática. Por esta razón, la computadora de destino no necesita tener CMake, Visual Studio, Blender ni el repositorio; únicamente requiere Windows de 64 bits y una tarjeta gráfica con un controlador compatible con OpenGL 3.3 Core o superior.

Al concluir el flujo, GitHub Actions publica temporalmente el instalador y su suma SHA-256 como un artefacto. Para conservar una descarga estable se creó la versión `v1.0.0`, titulada *CEMPASÚCHIL v1.0.0 — Entrega final*, dentro de GitHub Releases. La página incluye `CEMPASUCHIL-Setup-0.2.0-win64.exe`, el archivo `SHA256.txt`, los requisitos y las instrucciones completas de uso.

Haz clic aquí para descargar la versión más reciente de CEMPASÚCHIL

La instalación se resume en los siguientes pasos:

1. Descargar el archivo `CEMPASUCHIL-Setup-0.2.0-win64.exe` desde la sección *Assets* de la versión más reciente.
2. Ejecutar el instalador y seguir las indicaciones del asistente. Al ser un proyecto académico sin firma digital, Windows puede mostrar una advertencia antes de abrirlo.
3. Iniciar *CEMPASÚCHIL* desde el menú Inicio o desde el acceso directo creado por el instalador.

El instalador contiene todos los recursos de la escena y añade un desinstalador. El archivo `README.txt`, instalado junto con la aplicación, reúne los controles, las animaciones, las opciones de línea de comandos y la solución de problemas. No es necesario descargar el código fuente ni copiar por separado las carpetas de recursos.

5.3. Control de versiones e integración del trabajo

El control de versiones fue uno de los retos más constantes. El equipo trabajó desde macOS, Windows y Linux, y cada sistema generaba carpetas, proyectos, cachés y archivos temporales diferentes. En las primeras iteraciones algunos productos de CMake, Xcode o Visual Studio llegaron a mezclarse con el código fuente. Esto aumentaba el tamaño de los cambios, producía

conflictos que no representaban trabajo real y hacía difícil distinguir un recurso necesario de un archivo regenerable.

La solución consistió en definir una estructura común y fortalecer `.gitignore`: las carpetas `build-*`, binarios, objetos, registros, cachés, respaldos de Blender y metadatos del sistema operativo quedaron fuera del repositorio. Los modelos fuente, texturas finales, scripts de exportación, código, shaders y documentación sí permanecieron versionados. Cuando el repositorio remoto y el local contenían cambios independientes se utilizó `git pull –rebase` para colocar los cambios locales sobre la historia remota, resolver conflictos de manera explícita y evitar un historial de fusiones accidentales.

Otro problema fue el peso de los recursos. Los archivos de Blender, texturas de alta resolución y cachés de animación no se comportan como código: Git no puede representar sus diferencias de forma eficiente. Antes de la entrega final se auditó qué cargaba realmente el programa. Se eliminaron compilaciones locales, registros, duplicados de fotografías, modelos sin uso y texturas asociadas, pero se conservaron los fuentes de Blender necesarios para demostrar la autoría y regenerar las animaciones. La limpieza se verificó con una compilación desde cero y una captura automática.

5.4. Problemas de rendimiento y decisiones de optimización

La zona del altar fue el principal punto de presión: concentra miles de pétalos, decenas de veladoras, flores, alimentos, recipientes transparentes, papel picado y varias fuentes de luz. La primera implementación perdía cuadros al acercarse porque combinaba mallas densas con demasiadas evaluaciones de luz por objeto. No se optó por hornear luz estática, ya que las veladoras y los faroles debían conservar interacción en OpenGL.

Se aplicaron medidas complementarias: reducción de polígonos en modelos pesados, mapas normales para conservar detalle aparente, dibujo instanciado, descarte por *frustum*, cajas envolventes para filtrar luces, agrupación de fuentes del altar, caché de materiales y actualización de búferes solamente cuando una animación lo requiere. El resultado conserva la iluminación dinámica y la composición visual, pero evita calcular todas las luces contra todos los vértices o fragmentos de la escena.

5.5. Validación de las animaciones

Las animaciones se probaron en Linux sobre Mesa (`llvmpipe`, OpenGL 4.5 Core) dentro de un servidor X virtual. Un *script* externo al repositorio abría la aplicación en una vista fija, enviaba

las teclas con `xdotool` y tomaba capturas a intervalos con FFmpeg. Así se comprobó cada animación sin intervención manual. Las capturas de la sección de resultados provienen de esas corridas. Se verificó además:

- que las pistas exportadas tuvieran el tamaño esperado: 264 cuadros $\times$ 720 pétalos $\times$ 8 valores para el remolino, y 240 cuadros $\times$ 13 valores para la jarra;
- que la jarra arrancara a 18 cm del vaso, se inclinara 60° durante el vertido y que el vaso pasara de 3 % a 82 % de llenado;
- que cada personaje tuviera siete articulaciones y todos sus clips, con la duración en cuadros definida en Blender;
- que el pan del visitante 1 quedara apoyado en el hueco frontal del primer piso, sin intersecar las calaveritas ni las veladoras.

5.6. Rendimiento

Como el motor ya filtraba luces por caja envolvente y descartaba por *frustum*, las animaciones nuevas se diseñaron para no alterar esos mecanismos. El remolino actualiza un solo búfer de instancias por cuadro. Los personajes deforman en CPU mallas de pocos cientos de vértices. La jarra solo recalcula el tubo del chorro mientras está visible. Ninguna animación agrega llamadas de dibujo por elemento animado.

6. Resultados

El resultado final de *CEMPASÚCHIL* es una escena tridimensional recorrible que reúne las distintas etapas trabajadas durante el semestre: planeación, modelado, composición, texturizado, iluminación, animación, interacción y optimización. La aplicación dejó de ser solamente una representación aislada de un altar y se convirtió en un entorno completo, formado por el panteón, el camino de pétalos, la casa de adobe y la ofrenda. Esta distribución permite que el usuario no vea todos los elementos desde el inicio, sino que los descubra mientras avanza desde la entrada hasta el altar.

La escena final quedó compuesta por 2 134 nodos, 81 modelos cargados y 10 549 instancias. La diferencia entre la cantidad de modelos y la cantidad de elementos visibles se debe principalmente a la reutilización de geometría. Los pétalos, flores, veladoras, mariposas, bardas y otros objetos repetidos comparten sus mallas y se distribuyen mediante transformaciones independientes. Esto permitió poblar el entorno sin almacenar una copia completa de cada elemento y sin cambiar la composición visual que ya se había aprobado.

El altar fue la zona que recibió una mayor cantidad de trabajo. Además de sus tres niveles, se integraron veladoras, alimentos, pan de muerto, dulces, bebidas, recipientes, calaveritas, flores, papel picado, jarrones, fotografías y distintos objetos decorativos. La colocación de cada pieza tuvo que revisarse varias veces porque los modelos provenían de archivos distintos y no siempre compartían la misma escala, orientación u origen. En Blender algunos objetos se veían correctamente de manera individual, pero al exportarlos podían aparecer demasiado grandes, girados, enterrados en la superficie o separados de su punto de apoyo.

Para resolver esta parte no fue suficiente con mover los objetos hasta que se vieran bien desde una sola cámara. También se revisó que no atravesaran las veladoras, que los alimentos quedaran apoyados sobre los niveles, que los portarretratos no quedaran ocultos por los jarrones y que la composición siguiera siendo legible desde distintos ángulos. La ofrenda terminó funcionando como una composición completa y no como una acumulación de modelos independientes.

La integración de las fotografías permitió cerrar el propósito visual del altar. Los siete retratos corresponden a Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix y Kristen Pfaff. En lugar de duplicar todos los modelos de portarretrato, se reutilizaron tres modelos base y se generaron variantes de sus materiales para asignar una imagen diferente a cada posición. De esta forma se conservaron la geometría, el marco y sus materiales, modificando solamente la superficie donde se presenta la fotografía.

En la etapa de texturizado se comprobó que la apariencia conseguida en Blender no se trasladaba automáticamente a OpenGL. Varias redes de nodos utilizaban procedimientos propios

de Blender que el formato glTF no podía reproducir de manera directa. Por esta razón, las texturas procedurales se hornearon en mapas de color, rugosidad y normales. El horneado permitió conservar detalles como irregularidades de la cerámica, relieve en el pan, desgaste en las superficies y variaciones de color sin aumentar demasiado la geometría.

También fue necesario revisar coordenadas UV, espacios de color, transparencias y orientación de mapas normales. Algunos errores sólo aparecían después de exportar: bordes oscuros en transparencias, texturas invertidas, materiales demasiado brillantes o imágenes que no cargaban fuera de macOS. La integración de distintos cargadores de imágenes y la reorganización de las rutas permitió que los materiales se encontraran de la misma forma en Windows, macOS y Linux.

La iluminación final se mantuvo completamente dentro de OpenGL. No se hornearon luces estáticas en las texturas porque las veladoras y los faroles debían reaccionar a las interacciones del usuario. La escena utiliza una iluminación fría para establecer el ambiente nocturno y luces cálidas para marcar el camino y destacar la ofrenda. El contraste entre ambas zonas ayudó a que el altar se convirtiera en el punto principal sin iluminar de manera uniforme todo el escenario.

La concentración de luces y modelos en el altar produjo uno de los principales problemas de rendimiento. Al acercarse a esa zona se combinaban miles de instancias, transparencias, materiales y fuentes puntuales. La solución no consistió en eliminar la iluminación, sino en reducir trabajo innecesario. Se optimizaron modelos pesados, se utilizaron mapas normales para conservar detalle, se agruparon luces cercanas, se filtraron fuentes mediante cajas envolventes y se descartaron objetos fuera de la cámara. Con estos cambios fue posible conservar el aspecto de la escena y reducir la pérdida de cuadros durante el recorrido.

La navegación también cambió respecto a las primeras entregas. La cámara dejó de funcionar únicamente como una cámara libre y pasó a simular un recorrido en primera persona. El usuario inicia en la entrada del panteón, a una altura cercana a la de una persona, y puede caminar con WASD. El movimiento incluye un balanceo ligero para evitar que el desplazamiento se sienta completamente rígido. El vuelo se mantuvo como una opción separada para inspeccionar la escena, pero durante la caminata no es posible subir o bajar accidentalmente.

La etapa de animación reunió tres formas distintas de trabajo. Las animaciones de la mariposa, el recorrido de cámara y las veladoras se desarrollaron directamente en código. El papel picado, el remolino de pétalos y la jarra se crearon en Blender y después se exportaron como datos que OpenGL pudiera reproducir. Finalmente, los visitantes y el esqueleto utilizaron armatures y animación por huesos. La escena cuenta así con nueve animaciones principales controladas por las teclas del 1 al 9, además de la interacción para encender y apagar los faroles.

La animación del papel picado fue una de las integraciones más complicadas porque una exportación convencional no conservaba exactamente la deformación observada en Blender. El movimiento se resolvió mediante una caché de vértices de 96 cuadros. OpenGL reproduce las posiciones exportadas y conserva fijo el borde superior unido al cable. Esto permitió que cada pieza se moviera de forma distinta y evitó que todos los papeles compartieran una deformación artificial.

En el remolino de pétalos se trabajó con 720 elementos animados. En lugar de crear una llamada de dibujo por pétalo, se actualiza un lote de instancias. La jarra, por otra parte, requirió coordinar la rotación del recipiente, el nivel del agua, el chorro y el llenado del vaso. Los personajes necesitaron cargar huesos, clips, pesos y matrices de transformación, además de mezclar poses para evitar cambios bruscos entre caminar, detenerse, colocar una ofrenda, saludar o bailar.

Otro resultado importante fue la compatibilidad entre plataformas. El proyecto se trabajó desde equipos con macOS, Windows y Linux, lo que generó diferencias en compiladores, rutas, bibliotecas y controladores gráficos. Algunas fallas parecían pertenecer a la escena, pero en realidad se originaban en la manera en que cada sistema encontraba los recursos o creaba el contexto de OpenGL. La configuración con CMake, los cargadores de imágenes y los lanzadores por plataforma ayudaron a mantener un mismo proyecto sin separar el código en versiones incompatibles.

El control de versiones también formó parte del desarrollo. El repositorio llegó a contener modelos de Blender, texturas, capturas, documentación y cachés de animación de gran tamaño. Los conflictos no aparecían solamente en el código; también surgían cuando dos integrantes modificaban archivos de composición, documentación o recursos binarios. Se tuvieron que reorganizar ramas, integrar cambios remotos mediante *rebase*, excluir carpetas generadas y revisar qué archivos eran realmente necesarios para la entrega.

La limpieza final redujo archivos duplicados, compilaciones locales, registros, modelos que ya no aparecían en la escena y versiones de fotografías que el programa no utilizaba. Al mismo tiempo, se conservaron los archivos fuente necesarios para demostrar el modelado y regenerar las animaciones. Después de la limpieza se realizó una compilación desde cero para comprobar que ningún recurso eliminado fuera indispensable.

En conjunto, el resultado cumple los requisitos de modelos propios y de librería, texturizado, skybox, interacción con luces, animaciones por código, animaciones externas, animaciones por huesos e interacción mediante entrada del usuario. Más allá del número de elementos, el principal resultado fue conseguir que trabajos realizados en momentos y herramientas diferentes funcionaran como una misma escena. El modelado de Blender, las texturas horneadas, la iluminación de OpenGL, las pistas de animación y la navegación quedaron integrados den-

tro de una aplicación que puede recorrerse y demostrarse como un producto completo.

7. Evidencias visuales de la escena final

Las siguientes capturas documentan la versión final de CEMPASÚCHIL. Se agrupan por composición, detalle, iluminación y animación para mostrar tanto el resultado visual como los cambios de estado que no pueden apreciarse en una sola imagen.

Figura 6: Vistas generales y detalles de los modelos. El recorrido conecta la entrada, el sendero y la ofrenda; los primeros planos muestran texturas horneadas, normales, transparencia y las fotografías del altar.

Figura 7: Iluminación interactiva y animaciones programadas en OpenGL. La tecla L modifica faroles y veladoras; las mariposas combinan Catmull–Rom con una jerarquía cuerpo–alas, y la cámara automática recorre la escena hasta el altar.

Papel en reposo

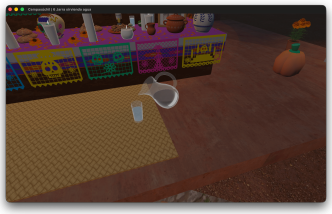
Deformación por viento

Inicio del remolino

Elevación de pétalos

Espiral formada

Inicio del vertido



Chorro y llenado del vaso

Figura 8: Animaciones creadas en Blender. Las cachés y pistas exportadas conservan la deformación del papel picado, el remolino de pétalos y el movimiento de la jarra con el nivel de agua.

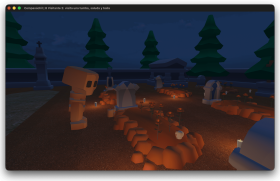
Llegada al altar

Preparación

Colocación de la ofrenda

Reverencia final

Recorrido entre tumbas



Observación de la tumba

Baile del visitante

Primer paso del esqueleto

Segundo paso del esqueleto

Figura 9: Animaciones por huesos. Los clips muestran al visitante que deposita un pan de muerto, al personaje que recorre el panteón y al esqueleto que baila junto a la portada.

8. Video demostrativo

Como parte de la entrega final se realizó un video demostrativo en el que se presenta el recorrido completo por la escena, los modelos principales, la iluminación dinámica, los controles de navegación y las nueve animaciones integradas en el proyecto.

El video muestra las tres animaciones desarrolladas mediante código, las tres animaciones creadas en Blender y las tres animaciones por huesos. También se incluye la interacción para encender y apagar los faroles del camino.

Figura 10: Video demostrativo de CEMPASÚCHIL. Se puede hacer clic sobre la imagen para abrir el video en YouTube.

Haz clic aquí para ver el video demostrativo

`https://youtu.be/2iYnnTLFxNo`

9. Conclusiones generales

CEMPASÚCHIL demuestra que una escena académica puede crecer de forma incremental sin desechar sus primeras decisiones. El altar de primitivas cumplió su función como herramienta de planeación; la fase de modelado convirtió esa idea en un espacio recorrible; el texturizado

y la iluminación establecieron una identidad visual; y la animación transformó los objetos estáticos en acontecimientos que el usuario puede activar y observar.

El mayor aprendizaje no fue una técnica aislada, sino la integración. Un modelo visualmente correcto en Blender todavía necesita escala, origen, UV, materiales exportables y un costo razonable para funcionar en OpenGL. Una luz atractiva puede volverse demasiado cara cuando se multiplica por miles de instancias. Una animación convincente en software externo requiere decidir qué datos exportar y cómo reproducirlos. Una aplicación que funciona en una computadora necesita rutas, decodificadores y lanzadores portables para convertirse en una entrega reproducible.

La optimización no se trató de retirar detalle indiscriminadamente. Se conservaron la iluminación dinámica, los materiales translúcidos, las fotografías, la densidad visual del altar y las animaciones, pero se cambió la forma de calcularlos: instancias en lugar de copias, mapas normales en lugar de geometría innecesaria, luces filtradas en lugar de evaluaciones globales y pistas compactas en lugar de depender de Blender durante la ejecución.

Finalmente, la ofrenda da una razón humana al trabajo técnico. Los retratos de Janis Joplin, Kurt Cobain, Valentín Elizalde, Amy Winehouse, Jim Morrison, Jimi Hendrix y Kristen Pfaff convierten el altar en un espacio de memoria. El recorrido no termina solamente en la zona con mayor cantidad de modelos o luces; termina frente a siete vidas breves cuyas obras continúan acompañando a otras personas. Esa permanencia es la idea que une el nombre del proyecto, el camino de cempasúchil y el movimiento final de la escena.

10. Conclusiones individuales

Basilio Illescas Andrés

Al inicio parecía que el proyecto podía resolverse construyendo un altar y colocando algunos objetos representativos, pero desde el prototipo se notó que la distribución iba a ser tan importante como el detalle de los modelos. Trabajar primero con primitivas ayudó a definir los tres niveles y a identificar qué objetos necesitaban mayor espacio. Cuando la propuesta creció hacia el panteón, el camino y la casa, también fue necesario cambiar la manera de organizar el trabajo, porque ya no se trataba de una sola composición sino de varias zonas conectadas.

Uno de los retos principales fue mantener una dirección visual común cuando los modelos se desarrollaban por separado. Cada objeto podía verse correcto en Blender, pero al reunirlos aparecían diferencias de escala, cantidad de detalle y estilo. Fue necesario revisar varias veces

la proporción del altar, la separación entre objetos y la forma en que se leía la escena desde el recorrido. La planeación no terminó en la primera entrega; se tuvo que ajustar cada vez que aparecían nuevos modelos, animaciones o problemas de rendimiento.

Desde el papel de Product Owner, la parte más importante fue decidir qué elementos realmente aportaban al concepto y cuáles sólo agregaban complejidad. La cantidad de objetos creció bastante, pero se procuró que la escena siguiera teniendo un recorrido reconocible y que el altar mantuviera la atención principal. También se revisó que las entregas parciales no quedaran como proyectos separados, sino que cada una sirviera como base para la siguiente.

El resultado final conserva la idea del primer prototipo, aunque visualmente sea mucho más grande y detallado. La estructura de tres niveles, la luz cálida y los retratos continúan siendo el centro de la propuesta. Considero que el mayor avance fue convertir una idea sencilla en un entorno completo sin perder el propósito original de la ofrenda. Este proyecto también dejó claro que planear no significa fijar todo desde el inicio, sino revisar continuamente si lo que se está construyendo sigue funcionando como conjunto.

Cruz Macedo Samuel Santiago

Mi participación estuvo relacionada principalmente con los elementos arquitectónicos y ambientales, por lo que uno de los primeros retos fue conseguir que el panteón, la entrada, el camino y la casa se sintieran como partes de un mismo espacio. Al trabajar con modelos propios y modelos de librería, las diferencias de escala y orientación aparecieron desde las primeras integraciones. No bastaba con importar los objetos; había que corregir su posición, revisar el terreno y observarlos desde la altura real de la cámara.

La compatibilidad entre Windows, Linux y los demás equipos del grupo también produjo varios problemas. Hubo casos en los que el proyecto compilaba, pero no encontraba las texturas o los shaders porque se estaba ejecutando desde una carpeta distinta. En otras ocasiones la dificultad estaba relacionada con Visual Studio, las versiones de CMake o los controladores de OpenGL. Esto obligó a separar los errores del código de los errores propios del entorno de ejecución.

El trabajo con GitHub fue otra parte importante. Al manejar modelos, texturas, capturas y documentación, los cambios podían ser mucho más pesados que un proyecto formado solamente por código. También aparecieron diferencias entre la versión local y la remota, conflictos durante la integración y archivos generados por cada sistema operativo que no debían formar parte del repositorio. Aprendí que un buen uso de Git no consiste únicamente en hacer commits, sino en mantener cambios claros, revisar el estado antes de sincronizar y evitar agregar archivos que pueden regenerarse.

Las pruebas en distintas plataformas ayudaron a encontrar supuestos que no eran visibles en un solo equipo. Las rutas, los formatos de imagen, la creación de la ventana y la disponibilidad de OpenGL podían cambiar aunque el código fuera el mismo. Considero que esta parte mejoró la estabilidad del proyecto porque nos obligó a preparar una aplicación menos dependiente de una computadora específica. El resultado no sólo fue una escena completa, sino un proyecto que puede compilarse y revisarse fuera del entorno donde se desarrolló originalmente.

Pérez Olvera Alexis Abraham

Una parte considerable de mi trabajo se concentró en los modelos y objetos de la ofrenda. Al principio el objetivo era cumplir con la cantidad de modelos solicitada, pero conforme el altar se llenó fue más importante revisar cómo convivían entre ellos. Algunos objetos ocultaban a otros, atravesaban las veladoras o no quedaban apoyados correctamente. La composición final necesitó varios ajustes para distribuir alimentos, recipientes, flores, retratos y elementos decorativos sin perder claridad.

El texturizado presentó dificultades distintas al modelado. Los materiales que se veían bien en Blender no siempre podían exportarse de la misma manera, especialmente cuando utilizaban nodos procedurales. El horneado de texturas permitió trasladar parte de ese detalle a OpenGL, pero también exigió revisar UV, resoluciones, espacios de color y mapas normales. En algunos casos fue necesario regresar al modelo, corregir su mapeo y volver a exportarlo antes de obtener un resultado aceptable.

La iluminación cambió completamente la percepción de los materiales. Una textura podía parecer correcta con la luz de Blender y demasiado oscura o brillante en la aplicación. Los objetos de vidrio y agua necesitaron un tratamiento diferente a los materiales opacos, mientras que las veladoras y los faroles debían iluminar sin volver demasiado costoso el renderizado. Esto hizo evidente que modelado, texturizado e iluminación no pueden revisarse de forma independiente.

La integración de las fotografías fue una de las partes que terminó de darle sentido al altar. Tener siete imágenes diferentes sin duplicar toda la geometría de los marcos permitió conservar una organización más limpia. También fue necesario cuidar el encuadre y la orientación de cada imagen para que se viera correctamente sobre la superficie del modelo. Al ver la ofrenda completa, considero que el trabajo de detalle sí ayudó a que los objetos dejaran de sentirse como pruebas separadas y comenzaran a formar una escena con intención.

Trejo Olvera Emmanuel

Mi trabajo se enfocó principalmente en la integración general, el desarrollo en OpenGL, la optimización y las pruebas en macOS. Uno de los problemas más frecuentes fue que algo podía funcionar correctamente en Blender o en una prueba aislada, pero no necesariamente dentro de la escena completa. Integrar un modelo implicaba revisar su ruta, materiales, escala y caja envolvente; integrar una animación requería además coordinar tiempos, transformaciones y actualización de buffers sin afectar el resto del renderizado.

En macOS aparecieron varios detalles relacionados con las rutas de ejecución, el escalado de pantalla Retina, la carga de imágenes y el comportamiento del lanzador. En algunos casos la ventana se creaba correctamente, pero el área de renderizado no coincidía con el tamaño real del framebuffer. También fue necesario considerar que Finder, la terminal y una aplicación empaquetada no utilizan siempre el mismo directorio de trabajo. Esto llevó a mejorar la localización de recursos y a dejar de depender de rutas relativas a la carpeta desde la que se ejecutaba el programa.

La integración de las animaciones fue probablemente la parte más complicada. La mariposa y el recorrido podían controlarse directamente con código, pero el papel picado debía conservar el movimiento creado en Blender. Las primeras aproximaciones no producían el mismo resultado: los papeles se movían de manera uniforme o se deformaban desde zonas incorrectas. Exportar una caché de vértices permitió mantener fijo el borde superior y reproducir la deformación de cada pieza. El remolino, la jarra y los personajes presentaron retos similares porque cada uno requería representar información distinta dentro del motor.

El rendimiento también se volvió un problema al llegar al altar. La cantidad de instancias, transparencias y luces provocaba una pérdida de cuadros que no se percibía en otras zonas. No se quería eliminar la iluminación ni reducir demasiado la composición, por lo que se trabajó en filtrar luces, reutilizar geometría, descartar elementos fuera de cámara y optimizar modelos pesados. Esta parte requirió probar muchas veces la escena completa, porque una mejora local podía causar un cambio visual en otro material o animación.

Finalmente, la integración remota y GitHub obligaron a tener cuidado con archivos grandes, ramas y versiones estables. Hubo momentos en los que la documentación, el código y los recursos avanzaban al mismo tiempo, por lo que era fácil mezclar cambios o intentar subir archivos generados demasiado pesados. Mantener una rama estable, revisar cada integración y limpiar el proyecto antes de empaquetarlo fue tan importante como implementar las funciones gráficas. Lo que más me deja el proyecto es entender que una escena de este tamaño no se termina solamente cuando se ve bien, sino cuando también puede compilarse, ejecutarse, documentarse y entregarse de manera consistente.

11. Referencias

- Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M. y Hillaire, S. (2018). *Real-time rendering* (4.ª ed.). CRC Press.
- Barry, P. J. y Goldman, R. N. (1988). A recursive evaluation algorithm for a class of Catmull-Rom splines. *ACM SIGGRAPH Computer Graphics, 22*(4), 199–204. https://doi.org/10.1145/378456.378511
- Blender Foundation. (2026). *Blender 5.2 manual*. https://docs.blender.org/manual/es/latest/
- Catmull, E. y Rom, R. (1974). A class of local interpolating splines. En R. E. Barnhill y R. F. Riesenfeld (Eds.), *Computer aided geometric design* (pp. 317–326). Academic Press.
- Gregory, J. (2018). *Game engine architecture* (3.ª ed.). CRC Press.
- Gribb, G. y Hartmann, K. (2001). *Fast extraction of viewing frustum planes from the world-view-projection matrix*. https://www.gamedevs.org/uploads/fast-extraction-viewing-frustum-planes-from-world-view-projection-matrix.pdf
- Kenney. (2021). *Graveyard kit* [Modelos 3D, licencia CC0]. https://kenney.nl/assets/graveyard-kit
- Khronos Group. (2021). *glTF 2.0 specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html
- Magnenat-Thalmann, N., Laperrière, R. y Thalmann, D. (1988). Joint-dependent local deformations for hand animation and object grasping. En *Proceedings of Graphics Interface '88* (pp. 26–33). Canadian Information Processing Society.
- Phong, B. T. (1975). Illumination for computer generated pictures. *Communications of the ACM, 18*(6), 311–317. https://doi.org/10.1145/360825.360839
- Schlick, C. (1994). An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum, 13*(3), 233–246. https://doi.org/10.1111/1467-8659.1330233
- Segal, M. y Akeley, K. (2010). *The OpenGL graphics system: A specification (version 3.3, core profile)*. Khronos Group. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- Shoemake, K. (1985). Animating rotation with quaternion curves. *ACM SIGGRAPH Computer Graphics, 19*(3), 245–254. https://doi.org/10.1145/325165.325242

- UNESCO. (2008). *La festividad indígena dedicada a los muertos*. Patrimonio Cultural Inmaterial. https://ich.unesco.org/es/RL/la-festividad-indigena-dedicada-a-los-muertos-00054
- Yuksel, C., Schaefer, S. y Keyser, J. (2011). Parameterization and applications of Catmull-Rom curves. *Computer-Aided Design, 43*(7), 747–755. https://doi.org/10.1016/j.cad.2010.08.008